

4

Control Flow and Collection Types

One of the central duties of a computer is to control what happens when predetermined conditions are met. When you click on a folder, you expect it to open; when you type on the keyboard, you expect the text to mirror your keystrokes. Writing C# code for applications or games is no different—they both need to behave in a certain way in one state, and in another when conditions change. In programming terms, this is called control flow, which is apt because it controls the flow of how code is executed in different scenarios.

In addition to working with control statements, we'll be taking a hands-on look at collection data types. Collections are a category of types that allow multiple values, and groupings of values, to be stored in a single variable. We'll break the chapter down into the following topics:

- Selection statements
- Working with array, dictionary, and list collections
- Iteration statements with `for`, `foreach`, and `while` loops
- Fixing infinite loops

Selection statements

The most complex programming problems can often be boiled down to sets of simple choices that a game or program evaluates and acts on. Since Visual Studio and Unity can't make those choices by themselves, writing out those decisions is up to you.

The if-else and switch selection statements allow you to specify branching paths, based on one or more conditions, and the actions you want to be taken in each case. Traditionally, these conditions include the following:

- Detecting user input
- Evaluating expressions and Boolean logic
- Comparing variables or literal values

You're going to start with the simplest of these conditional statements, if-else, in the following section.

The if-else statement

if-else statements are the most common way of making decisions in code. When stripped of all its syntax, the basic idea behind an if-else statement is: *If my condition is met, execute this block of code; if it's not, execute this other block of code.* Think of if-else statements as gates, or doors, with the conditions as their keys. To pass through, the key needs to be valid. Otherwise, entry will be denied and the code will be sent to the next possible door. Let's take a look at the syntax for declaring one of these gates.

A valid if-else statement requires the following:

- The if keyword at the beginning of the line
- A pair of parentheses to hold the condition
- A statement body inside curly brackets

It looks like this:

```
if(condition is true)
{
    Execute block of code
}
```

Optionally, an else statement can be added to store the action you want to take when the if statement condition fails. The same rules apply for the else statement:

```
else
{
    Execute another block of code
}
```

In blueprint form, the syntax almost reads like a sentence, which is why this is the recommended approach:

```
if(condition is true)
{
    Execute this code
    block
}
else
{
    Execute this code
    block
}
```

Since these are great introductions to logical thinking, at least in programming, we'll break down the three different if-else variations in more detail. Adding this code to your `LearningCurve.cs` script is optional right now, as we'll get into more details in later exercises:

1. A single if statement can exist by itself in cases where you don't care about what happens if the condition isn't met. In the following example, if `HasDungeonKey` is set to `true`, then a debug log will print out; if set to `false`, no code will execute:

```
public class LearningCurve: MonoBehaviour
{
    public bool HasDungeonKey = true;

    void Start()
    {
        if(hasDungeonKey)
        {
            Debug.Log("You possess the sacred key: enter.");
        }
    }
}
```



When referring to a condition as being met, I mean that it evaluates to `true`, which is often referred to as a passing condition.

2. Add an else statement in cases where an action needs to be taken whether the condition is true or false. If `HasDungeonKey` were false, the if statement would fail and the code execution would jump to the else statement:

```
public class LearningCurve: MonoBehaviour
{
    public bool HasDungeonKey = true;

    void Start()
    {
        if(HasDungeonKey)
        {
            Debug.Log("You possess the sacred key: enter.");
        }
        else
        {
            Debug.Log("You have not proved yourself yet.");
        }
    }
}
```

3. For cases where you need to have more than two possible outcomes, add an else-if statement with its parentheses, conditions, and curly brackets. This is best shown rather than explained, which we'll do in the next exercise.

Keep in mind that if statements can be used by themselves, but the other statements cannot exist on their own. You can also create more complex conditions with basic math operations, such as:

- > (greater than)
- < (less than)
- >= (greater than or equal to)
- <= (less than or equal to)
- == (equivalent)

For example, a condition of `(2 > 3)` will return false and fail, while a condition of `(2 < 3)` will return true and pass. Let's write out an if-else statement that checks the amount of money in a character's pocket, returning different debug logs for three different cases—greater than 50, less than 15, and anything else:

1. Open up `LearningCurve` and add a new public `int` variable named `CurrentGold` to the top of the script and set its value to between 1 and 100:

```
public int CurrentGold = 32;
```

2. Create a public method with no return value, called `Thievery`:

```
public void Thievery()  
{  
}
```

3. Inside the new function, add an `if` statement to check whether `CurrentGold` is greater than 50, and print a message to the console if this is true:

```
if(CurrentGold > 50)  
{  
    Debug.Log("You're rolling in it!");  
}
```

4. Add an `else-if` statement to check whether `CurrentGold` is less than 15 with a different debug log:

```
else if (CurrentGold < 15)  
{  
    Debug.Log("Not much there to steal...");  
}
```

5. Add an `else` statement with no condition and a final default log:

```
else  
{  
    Debug.Log("Looks like your purse is in the sweet spot.");  
}
```

6. Call the `Thievery` method inside `Start`:

```
void Start()  
{  
    Thievery();  
}
```

7. Save the file, check that your method matches the code below, and click **Play**:

```
public void Thievery()  
{  
    if(CurrentGold > 50)  
    {  
        Debug.Log("You're rolling in it!");  
    }  
    else if (CurrentGold < 15)  
    {  
        Debug.Log("Not much there to steal...");  
    }  
    else  
    {  
        Debug.Log("Looks like your purse is in the sweet spot.");  
    }  
}
```

With CurrentGold set to 32 in my example, we can break down the code sequence as follows:

1. The if statement and debug log are skipped because CurrentGold is not greater than 50.
2. The else-if statement and debug log are also skipped because CurrentGold is not less than 15.
3. Since 32 is not less than 15 or greater than 50, neither of the previous conditions was met, so the else statement executes and the third debug log is displayed:

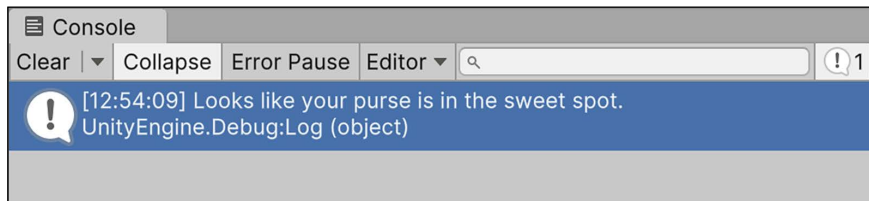


Figure 4.1: Screenshot of the console showing the debug output

To provide a complete view of the Unity editor, all our screenshots are taken in full-screen mode.

For color versions of all book images, use the link below: <https://packt.link/7yy5V>.

After trying out some other values for CurrentGold on your own, let's discuss what happens if we want to test a failing condition.

Using the NOT operator

Use cases won't always require checking for a positive, or true, condition, which is where the NOT operator comes in. Represented by typing a single exclamation point, the NOT operator allows negative, or false, conditions to be met by if or else-if statements. This means that the following conditions are the same:

```
if(variable == false)

// AND

if(!variable)
```

As you already know, you can check for Boolean values, literal values, or expressions in an if condition. So, naturally, the NOT operator has to be adaptable.

Take a look at the following example of two different negative values, `HasDungeonKey` and `WeaponType`, used in an if statement:

```
public class LearningCurve : MonoBehaviour
{
    public bool HasDungeonKey = false;
    public string WeaponType = "Arcane Staff";

    void Start()
    {
        if(!HasDungeonKey)
        {
            Debug.Log("You may not enter without the sacred key.");
        }

        if(WeaponType != "Longsword")
        {
            Debug.Log("You don't appear to have the right type of
weapon...");
        }
    }
}
```

We can evaluate each statement as follows:

- The first statement can be translated to, *If HasDungeonKey is false, the if statement evaluates to true and executes its code block.*

If you're asking yourself how a false value can evaluate to true, think of it this way: the if statement is not checking whether the value is true, but that the expression itself is true. HasDungeonKey might be set to false, but that's what we're checking for, so it's true in the context of the if condition.

- The second statement can be translated to, *If the string value of WeaponType is not equal to Longsword, then execute this code block.*

If you were to put this code into LearningCurve.cs, the results would match the following screenshot:

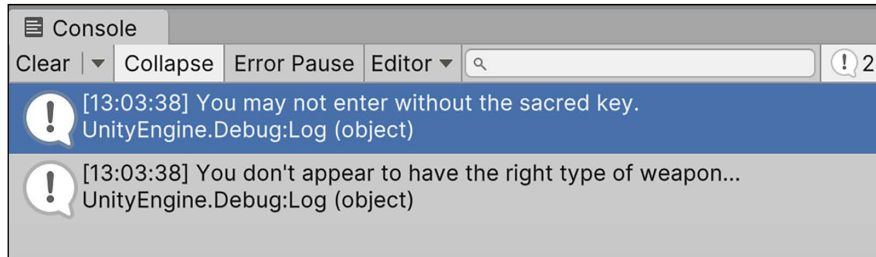


Figure 4.2: Screenshot of the console showing the NOT operator output

However, if you're still confused, copy the code we've looked at in this section into LearningCurve.cs and play around with the variable values until it makes sense.

So far, our branching conditions have been fairly simple, but C# also allows conditional statements to be nested inside each other for more complex situations.

Nesting statements

One of the most valuable functions of if-else statements is that they can be nested inside each other, creating complex logic routes through your code. In programming, we call them decision trees. Just like a real hallway, there can be doors behind other doors, creating a labyrinth of possibilities:

```
public class LearningCurve : MonoBehaviour
{
    public bool WeaponEquipped = true;
    public string WeaponType = "Longsword";
```

```
void Start()
{
    if(WeaponEquipped)
    {
        if(WeaponType == "Longsword")
        {
            Debug.Log("For the Queen!");
        }
    }
    else
    {
        Debug.Log("Fists aren't going to work against armor...");
    }
}
```

Let's break down the preceding example:

1. First, an `if` statement checks whether we have `WeaponEquipped`. At this point, the code only cares whether it's true, not what type of weapon it is.
2. The second `if` statement checks the `WeaponType` and prints out the associated debug log.
3. If the first `if` statement evaluates to false, the code would jump to the `else` statement and its debug log. If the second `if` statement evaluates to false, nothing is printed because there is no `else` statement.



The responsibility of handling logic outcomes is 100% on the programmer. It's up to you to determine the possible branches or outcomes your code can take.

What you've learned so far will get you through simple use cases with no problem. However, you'll quickly find yourself in need of more complex statements, which is where evaluating multiple conditions comes into play.

Evaluating multiple conditions

In addition to nesting statements, it's also possible to combine multiple condition checks into a single `if` or `else-if` statement with `AND` OR logic operators:

- `AND` is represented by two ampersand characters, `&&`. Any condition using the `AND` operator means that all conditions need to evaluate to `true` for the `if` statement to execute.
- `OR` is represented with two pipe characters, `||`. An `if` statement using the `OR` operator will execute if one or more of its conditions is `true`.
- Conditions are always evaluated from left to right.

In the following example, the `if` statement has been updated to check for both `WeaponEquipped` and `WeaponType`, both of which need to be `true` for the code block to execute:

```
if(WeaponEquipped && WeaponType == "Longsword")
{
    Debug.Log("For the Queen!");
}
```

The `AND` OR operators can be combined to check multiple conditions in any order. There is also no limit to how many operators you can combine—just be careful when using them together that you don't create logic conditions that will never execute.

It's time to put everything we've learned so far about `if` statements to the test. So, review this section if you need to, and then move on to the next section.

Let's cement this topic with a little treasure chest experiment:

1. Declare three variables at the top of `LearningCurve`: `PureOfHeart` is a `bool` and should be `true`, `HasSecretIncantation` is also a `bool` and should be `false`, and `RareItem` is a `string` and its value is up to you:

```
public bool PureOfHeart = true;
public bool HasSecretIncantation = false;
public string RareItem = "Relic Stone";
```

2. Create a public method at the bottom of the script with no return value called `OpenTreasureChamber`:

```
public void OpenTreasureChamber()
{
}
```

3. Inside `OpenTreasureChamber`, declare an if-else statement to check whether `PureOfHeart` is true *and* that `RareItem` matches the string value you assigned to it:

```
if(PureOfHeart && RareItem == "Relic Stone")
{
}
```

4. Create a nested if-else statement inside the first, checking whether `HasSecretIncantation` is false:

```
if(!HasSecretIncantation)
{
    Debug.Log("You have the spirit, but not the knowledge.");
}
```

5. Add debug logs for each if-else case.
6. Call the `OpenTreasureChamber` method inside `Start`:

```
void Start()
{
    OpenTreasureChamber();
}
```

7. Save, check that your code matches the code below, and click **Play**:

```
public class LearningCurve : MonoBehaviour
{
    public bool PureOfHeart = true;
    public bool HasSecretIncantation = false;
    public string RareItem = "Relic Stone";

    void Start()
    {
        OpenTreasureChamber();
    }

    public void OpenTreasureChamber()
    {
        if(PureOfHeart && RareItem == "Relic Stone")
        {
            if(!HasSecretIncantation)
```

```
    {
        Debug.Log("You have the spirit, but not the
knowledge.");
    }
    else
    {
        Debug.Log("The treasure is yours, worthy hero!");
    }
}
else
{
    Debug.Log("Come back when you have what it takes.");
}
}
```

💡 **Quick tip:** Enhance your coding experience with the **AI Code Explainer** and **Quick Copy** features. Open this book in the next-gen Packt Reader. Click the **Copy** button (1) to quickly copy code into your coding environment, or click the **Explain** button (2) to get the AI assistant to explain a block of code to you.



```
function calculate(a, b) {
    return {sum: a + b};
};
```

Copy

Explain

1

2

🔒 **The next-gen Packt Reader** is included for free with the purchase of this book. Unlock it by scanning the QR code below or visiting <https://www.packtpub.com/unlock/9781837636877>.



If you matched the variable values to the preceding screenshot, the nested `if` statement debug log will be printed out. This means that our code got past the first `if` statement checking for two conditions, but failed the third:

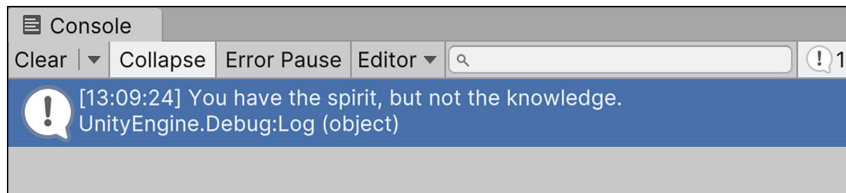


Figure 4.3: Screenshot of the debug output in the console

Now, you could stop here and use even bigger `if-else` statements for all your conditional needs, but that's not going to be efficient in the long run. Good programming is about using the right tool for the right job, which is where the `switch` statement comes in.

The switch statement

`if-else` statements are a great way to write decision logic. However, when you have more than three or four branching actions, they just aren't feasible. Before you know it, your code can end up looking like a tangled knot that's hard to follow, and a headache to update.

`switch` statements take in expressions and let us write out actions for each possible outcome, but in a much more concise format than `if-else`.

`switch` statements require the following elements:

- The `switch` keyword followed by a pair of parentheses holding its condition
- A pair of curly brackets
- A case statement for each possible path ending with a colon: individual lines of code or methods, followed by the `break` keyword and a semicolon
- A default case statement ending with a colon: individual lines of code or methods, followed by the `break` keyword and a semicolon

In blueprint form, it looks like this:

```
switch(matchExpression)
{
    case matchValue1:
        Executing code block
        break;
```

```
    case matchValue2:
        Executing code block
        break;
    default:
        Executing code block
        break;
}
```

The highlighted keywords in the preceding blueprint are the important bits. When a case statement is defined, anything between its colon and break keyword acts like the code block of an if-else statement. The break keyword just tells the program to exit the switch statement entirely after the selected case fires. Now, let's discuss how the statement determines which case gets executed, which is called pattern matching.

Pattern matching

In switch statements, pattern matching refers to how a **match expression** is validated against multiple case statements. A match expression can be of any type that isn't null or nothing; all case statement values need to match the type of the match expression.

For example, if we had a switch statement that was evaluating an integer variable, each case statement would need to specify an integer value for it to check against.

The case statement with a value that matches the expression is the one that is executed. If no case is matched, the default case fires. Let's try this out for ourselves!

That was a lot of new syntax and information, but it helps to see it in action. Let's create a simple switch statement for different actions a character could take:

1. Create a new public string variable named CharacterAction and set its value to "Attack":

```
public string CharacterAction = "Attack";
```

2. Create a public method with no return value called PrintCharacterAction:

```
public void PrintCharacterAction()
{
}
```

3. Declare a switch statement inside the new method and use `CharacterAction` as the match expression:

```
switch(CharacterAction)
{
}
```

4. Create two case statements for "Heal" and "Attack" with different debug logs. Don't forget to include the `break` keyword at the end of each:

```
case "Heal":
    Debug.Log("Potion sent.");
    break;
case "Attack":
    Debug.Log("To arms!");
    break;
```

5. Add a default case with a debug log and `break`:

```
default:
    Debug.Log("Shields up.");
    break;
```

6. Call the `PrintCharacterAction` method inside `Start`:

```
void Start()
{
    PrintCharacterAction();
}
```

7. Save the file, make sure your code matches the screenshot below, and click **Play**:

```
public string CharacterAction = "Attack";

void Start()
{
    PrintCharacterAction();
}

public void PrintCharacterAction()
{
    switch(CharacterAction)
```

```
{  
    case "Heal":  
        Debug.Log("Potion sent.");  
        break;  
    case "Attack":  
        Debug.Log("To arms!");  
        break;  
    default:  
        Debug.Log("Shields up.");  
        break;  
}
```

Since `CharacterAction` is set to "Attack", the switch statement executes the second case and prints out its debug log:

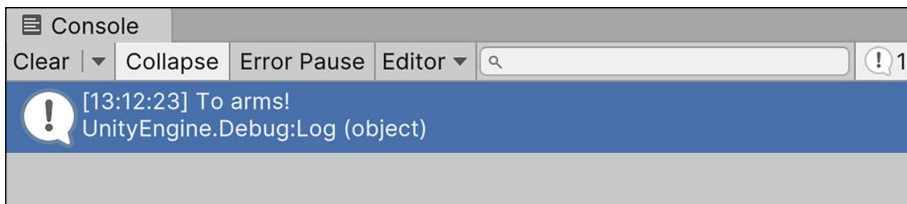


Figure 4.4: Screenshot of the switch statement output in the console



Change `CharacterAction` to either "Heal" or an undefined action to see the first and default cases in action.

There are going to be times where you need several, but not all, switch cases to perform the same action. These are called **fall-through cases** and are the subject of our next section.

Fall-through cases

switch statements can execute the same action for multiple cases, similar to how we specified several conditions in a single if statement. The term for this is called fall-through or, sometimes, fall-through cases. Fall-through cases let you define a single set of actions for multiple cases. If a case block is left empty or has code without the break keyword, it will fall through to the case directly beneath it. This helps keep your switch code clean and efficient, without duplicated case blocks.

Cases can be written in any order, so creating fall-through cases greatly increases code readability and efficiency.

Let's simulate a tabletop game scenario with a switch statement and fall-through case, where a dice roll determines the outcome of a specific action:

1. Create a public int variable named Dice and assign it a value of 7:

```
public int Dice = 7;
```

2. Create a public method with no return value called RollDice:

```
public void RollDice()  
{  
}  
}
```

3. Add a switch statement with Dice as the match expression:

```
switch(Dice)  
{  
}  
}
```

4. Add three cases for possible dice rolls at 7, 15, and 20, with a default case statement at the end.
5. Cases 15 and 20 should have their own debug logs and break statements, while case 7 should fall through to case 15:

```
case 7:  
case 15:  
    Debug.Log("Mediocre damage, not bad.");  
    break;  
case 20:  
    Debug.Log("Critical hit, the creature goes down!");  
    break;  
default:  
    Debug.Log("You completely missed and fell on your face.");  
    break;
```

6. Call the RollDice method inside Start:

```
void Start()
{
    RollDice();
}
```

7. Save the file and run it in Unity.



If you want to see the fall-through case in action, try adding a debug log to case 7, but without the break keyword.

With Dice set to 7, the switch statement will match with the first case, which will fall through and execute case 15 because it lacks a code block and a break statement. If you change Dice to 15 or 20, the console will show their respective messages, and any other value will fire off the default case at the end of the statement:

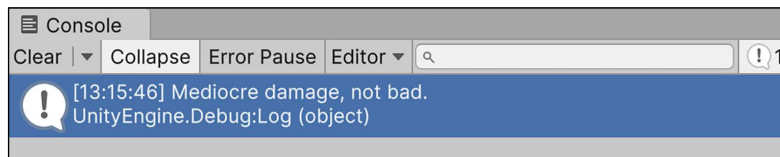


Figure 4.5: Screenshot of fall-through switch statement code



switch statements are extremely powerful and can simplify even the most complex decision logic. If you want to dig deeper into switch pattern matching, refer to: <https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/switch>.

That's all we need to know about conditional logic for the moment. So, review this section if you need to, and then test yourself on the following quiz before moving on to collections!

Pop Quiz 1—if, and, or but

Test your knowledge with the following questions:

1. What values are used to evaluate if statements?
2. Which operator can turn a true condition false or a false condition true?

3. If two conditions need to be true for an `if` statement's code to execute, what logical operator would you use to join the conditions?
4. If only one of two conditions needs to be true to execute an `if` statement's code, what logical operator would you use to join the two conditions?

Don't forget to check your answers against mine in the *Pop Quiz Answers* appendix to see how you did!

With that done, you're ready to step into the world of collection data types. These types are going to open up a whole new subset of programming functionality for your games and C# programs!

Collections at a glance

So far, we've only needed variables to store a single value, but there are many conditions where a group of values will be required. **Collection types** in C# include arrays, dictionaries, and lists—each has its strengths and weaknesses, which we'll discuss in the following sections.

Arrays

Arrays are the most basic collection that C# offers. Think of them as containers for a group of values, called *elements* in programming terminology, each of which can be accessed or modified individually:

- Arrays can store any type of value; all the elements need to be of the same type.
- The length, or the number of elements an array can have, is set when it's created and can't be modified afterward.
- If no initial values are assigned when it's created, each element will be given a default value. Arrays storing number types default to zero, while any other type gets set to null or nothing.

Arrays are the least flexible collection type in C#. This is mainly because elements can't be added or removed after they have been created. However, they are particularly useful when storing information that isn't likely to change. That lack of flexibility makes them faster compared to other collection types.

Declaring an array is similar to other variable types we've worked with, but has a few modifications:

- Array variables require a specified element type, a pair of square brackets, and a unique name.

- The new keyword is used to create the array in memory, followed by the value type and another pair of square brackets. The reserved memory area is the exact size of the data you're intending to store in the new array.
- The number of elements the array will store goes inside the second pair of square brackets.

In blueprint form, it looks like this:

```
elementType[] name = new elementType[numberOfElements];
```

Let's take an example where we need to store the top three high scores in our game:

```
int[] TopPlayerScores = new int[3];
```

Broken down, `TopPlayerScores` is an array of integers that will store three integer elements. Since we didn't add any initial values, each of the three values in `TopPlayerScores` is 0. However, if you change the array size, the contents of the original array are lost, so be careful.

You can assign values directly to an array when it's created by adding them inside a pair of curly brackets at the end of the variable declaration. C# has a longhand and shorthand way of doing this, but both are equally valid:

```
// Longhand initializer
int[] TopPlayerScores = new int[] {713, 549, 984};

// Shortcut initializer
int[] TopPlayerScores = { 713, 549, 984 };
```



Initializing arrays with the shorthand syntax is very common, so I'll be using it for the rest of the book. However, if you want to remind yourself of the details, feel free to use the explicit longhand initializer syntax as shown above.

Now that the declaration syntax is no longer a mystery, let's talk about how array elements are stored and accessed.

Indexing and subscripts

Each array element is stored in the order it's assigned, which is referred to as its **index**. Arrays are zero-indexed, meaning that the element order starts at 0 instead of 1. Think of an element's index as its reference, or location.

In `TopPlayerScores`, the first integer, 452, is located at index 0, 713 at index 1, and 984 at index 2:

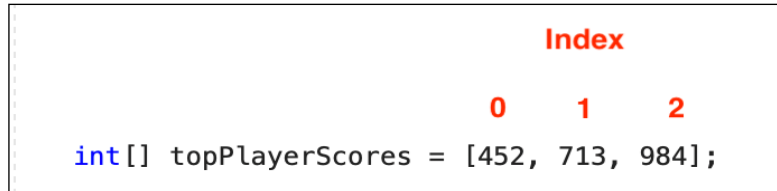


Figure 4.6: Array indexes mapped to their values

Individual values are located by their index using the **subscript operator**, which is a pair of square brackets that contains the index of the elements.

For example, to retrieve and store the second array element in `TopPlayerScores`, we would use the array name followed by subscript brackets and index 1:

```
// The value of score is set to 713
int score = TopPlayerScores[1];
```

The subscript operator can also be used to directly modify an array value just like any other variable, or even passed around as an expression by itself:

```
TopPlayerScores[1] = 1001;
```

The values in `TopPlayerScores` would then be 452, 1001, and 984.

Multidimensional arrays

Arrays are also a great way to store elements in a table format—think rows and columns in the real world. These are called multidimensional arrays because each added element brings another dimension to the data. The array examples above only hold one element per index, so they are one-dimensional. If we wanted an array to hold, say, an *x* and *y* coordinate in each element like in middle-school math class, we could create a two-dimensional array like so:

```
// The Coordinates array has 3 rows and 2 columns
int[,] Coordinates = new int[3,2];
```

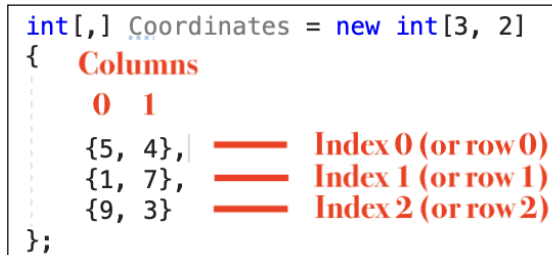
Notice we used a comma inside the square brackets to mark the array as two-dimensional, and we added two initialization fields, which are also separated by a comma.

We can also directly initialize a multidimensional array with values, so creating a table of x and y coordinates like the one above could be shortened to the following:

```
int[,] Coordinates = new int[3,2]
{
    {5,4},
    {1,7},
    {9,3}
};
```

You can see that we have three rows, or elements, each containing 2 columns of data for the x and y values. Now here's the tricky bit of mental gymnastics—you need to think of multidimensional arrays as arrays *of* arrays. In the above example, each element is still stored at an index starting with 0 and moving up, but each element is an array instead of a single value.

To put this in concrete terms, the value of 4 in the first column of the first row is located at index 0, and the actual value of 4 is located at the second element in that row's array, or 1:



```
int[,] Coordinates = new int[3, 2]
{
    Columns
    0 1
    {5, 4}, — Index 0 (or row 0)
    {1, 7}, — Index 1 (or row 1)
    {9, 3} — Index 2 (or row 2)
};
```

Figure 4.7: Multidimensional array mapped with indexes

In code, we would use the following subscript, using the row subscript first, followed by the column index:

```
// Finding the value in the first row, first column
int coordinateValue = Coordinates[0, 1];
```

Changing a value in a multidimensional array is the same as with a regular array, we use the subscript of the value we want to update and then assign a new value:

```
// Value in the first row, first column is now 10
Coordinates[0, 1] = 10;
```

A C# array can have up to 32 dimensions, which is a lot, but the rules for creating them are the same — add an extra comma for every dimension inside the type brackets at the beginning of the variable and an extra comma and number of elements in the initialization. For instance, a three-dimensional array would look like this:

```
int[, ,] Coordinates = new int[3,3,2];
```



This is a bit advanced for our needs, but you can get into more complex multidimensional array code at: <https://learn.microsoft.com/dotnet/csharp/programming-guide/arrays/multidimensional-arrays>.

Range exceptions

When arrays are created, the number of elements is set and unchangeable, which means we can't access an element that doesn't exist. In the `TopPlayerScores` example, the array length is 3, so the range of valid indices is from 0 to 2.

For example, if we tried to run the following code:

```
Debug.Log(TopPlayerScores[3]);
```

The console would print an aptly-named `IndexOutOfRangeException` error in the console because any index of 3 or higher is out of the array's range:

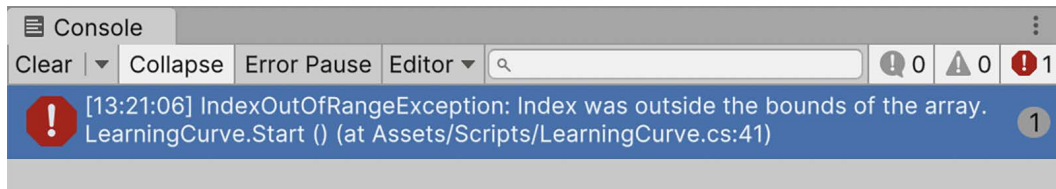


Figure 4.8: Screenshot of index-out-of-range exception

Good programming habits dictate that we avoid range exceptions by checking whether the value we want is within an array's index range, which we'll cover in the *Iteration statements* section.

You can always check the length of an array—that is, how many items it contains—with the `Length` property:

```
TopPlayerScores.Length;
```

Arrays aren't the only collection types C# has to offer. In the next section, we'll deal with lists, which are more flexible and more common in the programming landscape.

Lists

Lists are closely related to arrays, collecting multiple values of the same type in a single variable. They're much easier to deal with when it comes to adding, removing, and updating elements, but their elements aren't stored sequentially. They are also mutable, meaning you can change the length or number of items you're storing, without overwriting the whole variable. This can, sometimes, lead to a higher performance cost over arrays.



Performance cost refers to how much of a computer's time and energy a given operation takes up. Nowadays, computers are fast, but they can still get overloaded by big games or applications.

A list-type variable needs to meet the following requirements:

- The `List` keyword, its element type inside left and right arrow characters, and a unique name
- The `new` keyword to initialize the list in memory, with the `List` keyword and element type between arrow characters
- A pair of parentheses capped off by a semicolon

In blueprint form, it reads as follows:

```
List<elementType> name = new List<elementType>();
```



List length can always be modified, so there is no need to specify how many elements it will eventually hold when created.

Like arrays, lists can be initialized in the variable declaration by adding element values inside a pair of curly brackets:

```
List<elementType> name = new List<elementType>() { value1, value2 };
```

Elements are stored in the order they are added (instead of the sequential order of the values themselves), are zero-indexed like arrays, and can be accessed using the subscript operator.

Let's start setting up a list of our own to test out the basic functionality this class has on offer.

We'll start with a warm-up exercise by creating a list of party members in a fictional role-playing game:

1. Create a new `List` of the `string` type inside `Start` called `QuestPartyMembers`, and initialize it with the names of three characters:

```
List<string> QuestPartyMembers = new List<string>()
{
    "Grim the Barbarian",
    "Merlin the Wise",
    "Sterling the Knight"
};
```

2. Add a debug log to print out the number of party members in the list using the `Count` method:

```
Debug.LogFormat("Party Members: {0}", QuestPartyMembers.Count);
```

3. Save the file and play it in Unity.

We initialized a new list, called `QuestPartyMembers`, which now holds three string values, and used the `Count` method from the `List` class to print out the number of elements.

Notice that you use `Count` for lists, but `Length` for arrays.

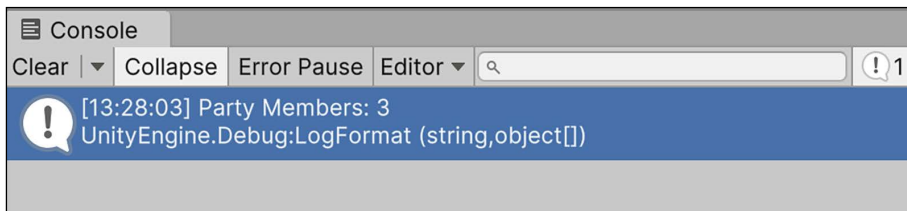


Figure 4.9: Screenshot of list item output in the console

Knowing how many elements are in a list is highly useful; however, in most cases, that information is not enough. We want to be able to modify our lists as needed, which we'll discuss next.

Accessing and modifying lists

List elements can be accessed and modified like arrays with a subscript operator and index, as long as the index is within the `List` class's range. However, the `List` class has a variety of methods that extend its functionality, such as adding, inserting, and removing elements.

Sticking with the `QuestPartyMembers` list, let's add a new member to the team:

```
QuestPartyMembers.Add("Craven the Necromancer");
```

The `Add()` method appends the new element at the end of the list, which brings the `QuestPartyMembers` count to four and the element order to the following:

```
{  
    "Grim the Barbarian",  
    "Merlin the Wise",  
    "Sterling the Knight",  
    "Craven the Necromancer"  
};
```

To add an element to a specific spot in a list, we can pass the index and the value that we want to add to the `Insert()` method:

```
QuestPartyMembers.Insert(1, "Tanis the Thief");
```

When an element is inserted at a previously occupied index, all the elements in the list have their indices increased by 1. In our example, "Tanis the Thief" is now at index 1, meaning that "Merlin the Wise" is now at index 2 instead of 1, and so on:

```
{  
    "Grim the Barbarian",  
    "Tanis the Thief",  
    "Merlin the Wise ",  
    "Sterling the Knight",  
    "Craven the Necromancer"  
};
```

Removing an element is just as simple; all we need is the index or the literal value, and the `List` class does the work:

```
// Both of these methods would remove the required element  
QuestPartyMembers.RemoveAt(0);  
QuestPartyMembers.Remove("Grim the Barbarian");
```

At the end of our edits, `QuestPartyMembers` now contains the following elements indexed from 0 to 3:

```
{  
    "Tanis the Thief",  
    "Merlin the Wise",  
    "Sterling the Knight",  
    "Craven the Necromancer"  
};
```

If you run the game now, you'll also see the party list length is 4 instead of 3!

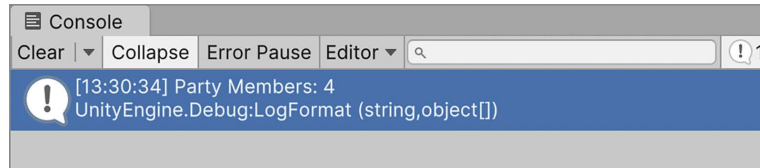


Figure 4.10: Screenshot of modified list items in the console



There are many more `List` class methods that allow for value checks, finding and sorting elements, and working with ranges. A full method list, with descriptions, can be found here: <https://docs.microsoft.com/en-us/dotnet/api/system.collections.generic.list-1?view=netframework-4.7.2>.

While lists are great for single-value elements, there are cases where you'll need to store information or data containing more than one value. This is where dictionaries come into play.

Dictionaries

The **Dictionary** type steps away from arrays and lists by storing value pairs in each element, instead of single values. These elements are referred to as key-value pairs: the key acts as the index, or lookup value, for its corresponding value. Unlike arrays and lists, dictionaries are unordered. However, they can be sorted and ordered in various configurations after they are created.

Declaring a dictionary is almost the same as declaring a list, but with one added detail—both the key and the value type need to be specified inside the arrow symbols:

```
Dictionary<keyType, valueType> name = new
Dictionary<keyType, valueType>();
```

To initialize a dictionary with key-value pairs, do the following:

- Use a pair of curly brackets at the end of the declaration
- Add each element within its pair of curly brackets, with the key and the value separated by a comma
- Separate elements with a comma, except the last element where the comma is optional

It looks like this:

```
Dictionary<keyType, valueType> name = new
Dictionary<keyType, valueType>()
{
```

```
{key1, value1},  
{key2, value2}  
};
```

An important note to consider when picking key values is that each key must be unique, and they cannot be changed. If you need to update a key, then you need to change its value in the variable declaration or remove the entire key-value pair and add another in the code, which we'll look at next.



Just like with arrays and lists, dictionaries can be initialized on a single line with no problems from Visual Studio. However, writing out each key-value pair on its line, as in the preceding example, is a good habit to get into—both for readability and your sanity.

Let's create a dictionary to store items that a character might carry:

1. Declare a Dictionary with a key type of `string` and a value type of `int` called `ItemInventory` in the `Start` method.
2. Initialize it to `new Dictionary<string, int>()`, and add three key-value pairs of your choice. Make sure each element is in its pair of curly brackets:

```
Dictionary<string, int> ItemInventory = new  
Dictionary<string, int>()  
{  
    { "Potion", 5 },  
    { "Antidote", 7 },  
    { "Aspirin", 1 }  
};
```

3. Add a debug log to print out the `ItemInventory.Count` property so that we can see how items are stored:

```
Debug.LogFormat("Items: {0}", ItemInventory.Count);
```

4. Save the file and play.

Here, a new dictionary, called `ItemInventory`, was created and initialized with three key-value pairs. We specified the keys as strings, with corresponding values as integers, and printed out how many elements `ItemInventory` currently holds:

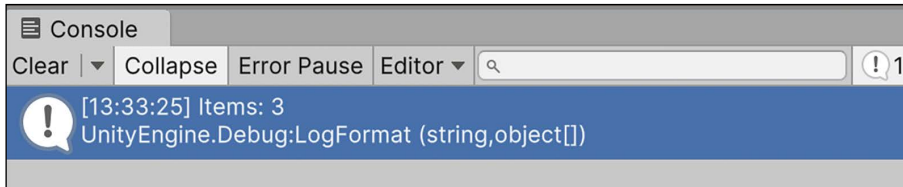


Figure 4.11: Screenshot of dictionary count in console

Like lists, we need to be able to do more than just print out the number of key-value pairs in a given dictionary. We'll explore adding, removing, and updating these values in the following section.

Working with dictionary pairs

Key-value pairs can be added, removed, and accessed from dictionaries using both subscript and class methods. To retrieve an element's value, use the subscript operator with the element's key—in the following example, `numberOfPotions` would be assigned a value of 5:

```
int numberOfPotions = ItemInventory["Potion"];
```

An element's value can be updated using the same method—the value associated with "Potion" would now be 10:

```
ItemInventory["Potion"] = 10;
```

Elements can be added to dictionaries in two ways: with the `Add` method and with the subscript operator. The `Add` method takes in a key and a value and creates a new key-value element, as long as their types correspond to the dictionary declaration:

```
ItemInventory.Add("Throwing Knife", 3);
```

If the subscript operator is used to assign a value to a key that doesn't exist in a dictionary, the compiler will automatically add it as a new key-value pair. For example, if we wanted to add a new element for "Bandage", we could do so with the following code:

```
ItemInventory["Bandage"] = 5;
```

This brings up a crucial point about referencing key-value pairs: it's better to be certain that an element exists before trying to access it, to avoid mistakenly adding new key-value pairs. Pairing the `ContainsKey` method with an `if` statement is the simple solution since `ContainsKey` returns a Boolean value based on whether the key exists. In the following example, we make sure that the "Aspirin" key exists using an `if` statement before modifying its value:

```
if(ItemInventory.ContainsKey("Aspirin"))
{
    ItemInventory["Aspirin"] = 3;
}
```

Finally, a key-value pair can be deleted from a dictionary using the `Remove()` method, which takes in a key parameter:

```
ItemInventory.Remove("Antidote");
```



Like lists, dictionaries offer a variety of methods and functionality to make development easier, but we can't cover them all here. If you're curious, the official documentation can be found at: <https://docs.microsoft.com/en-us/dotnet/api/system.collections.generic.dictionary-2?view=netframework-4.7.2>.

Collections are safely in our toolkit, so it's time for another quiz to make sure you're ready to move on to the next big topic: iteration statements.

Pop Quiz 2—all about collections

1. What is an element in an array or list?
2. What is the index number of the first element in an array or list?
3. Can a single array or list store different types of data?
4. How can you add more elements to an array to make room for more data?

Don't forget to check your answers against mine in the *Pop Quiz Answers* appendix to see how you did!

Since collections are groups or lists of items, they need to be accessible in an efficient manner. Luckily, C# has several iteration statements, which we'll talk about in the following section.

Iteration statements

We've accessed individual collection elements through the subscript operator, along with collection type methods, but what do we do when we need to go through the entire collection element by element? In programming, this is called **iteration**, and C# provides several statement types that let us loop through (or iterate over, if you want to be technical) collection elements. Iteration statements are like methods, in that they store a block of code to be executed; however, unlike methods, they can repeatedly execute their code blocks as long as their conditions are met.

for loops

The for loop is most commonly used when a block of code needs to be executed a certain number of times before the program continues. The statement itself takes in three expressions, each with a specific function to perform before the loop executes. Since for loops keep track of the current iteration, they are best suited to arrays and lists.

Take a look at the following looping statement blueprint:

```
for (initializer; condition; iterator)
{
    code block;
}
```

Let's break this down:

1. The `for` keyword starts the statement, followed by a pair of parentheses.
2. Inside the parentheses are the gatekeepers: the `initializer`, `condition`, and `iterator` expressions.
3. The loop starts with the `initializer` expression, which is a local variable created to keep track of how many times the loop has executed—this is usually set to `0` because collection types are zero-indexed.
4. Next, the `condition` expression is checked and, if true, proceeds to the `iterator`.
5. The `iterator` expression is used to either increase or decrease (**increment** or **decrement**) the `initializer`, meaning the next time the loop evaluates its condition, the `initializer` will be different.

Increasing and decreasing a value by 1 is called **incrementing** and **decrementing**, respectively (`--` will decrease a value by 1, and `++` will increase it by 1).

That all sounds like a lot, so let's look at a practical example with the `QuestPartyMembers` list we created earlier. Add the following code:

```
int listLength = QuestPartyMembers.Count;

for (int i = 0; i < listLength; i++)
{
    Debug.LogFormat("Index: {0} - {1}", i, QuestPartyMembers[i]);
}
```

Let's go through the loop again and see how it works:

1. First, the initializer in the for loop is set as a local `int` variable named `i` with a starting value of `0`.
2. Second, we store the list of the list in a variable so the loop doesn't need to check the length every time through, which is best practice for performance.
3. To ensure we never get an out-of-range exception, the for loop makes sure that the loop only runs another time if `i` is less than the number of elements in `QuestPartyMembers`:
 - With arrays, we use the `Length` property to determine how many items it has
 - With lists, we use the `Count` property
4. Finally, `i` is increased by 1 each time the loop runs with the `++` operator.
5. Inside the for loop, we've just printed out the index and the list element at that index using `i`.
6. Notice that `i` is in step with the index of the collection elements, since both start at `0`:

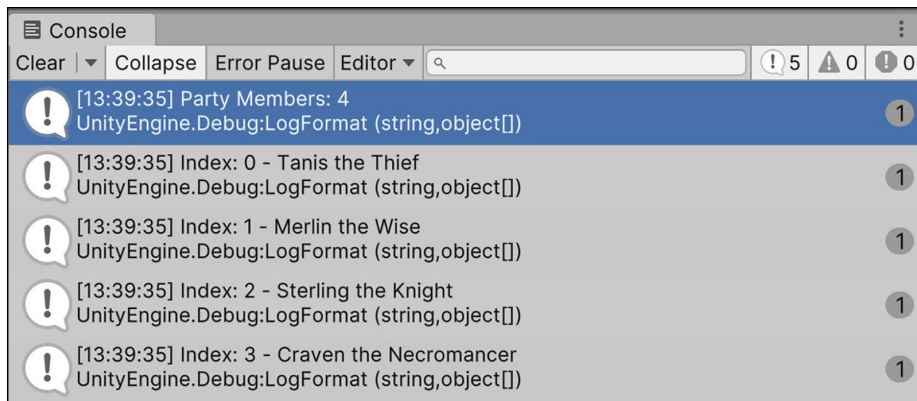


Figure 4.12: Screenshot of list values printed out with a for loop

Traditionally, the letter *i* is used as the initializer variable name. If you happen to have nested for loops, the variable names used should be the letters *j*, *k*, *l*, and so on.

Let's try out our new iteration statements on one of our existing collections.

While we loop through `QuestPartyMembers`, let's see whether we can identify when a certain element is iterated over and add a special debug log just for that case:

1. Move the `QuestPartyMembers` list and for loop into a public function called `FindPartyMember` and call it in `Start`.
2. Add an `if` statement below the debug log in the for loop to check whether the current `questPartyMember` list matches "Merlin the Wise":

```
if(QuestPartyMembers[i] == "Merlin the Wise")
{
    Debug.Log("Glad you're here Merlin!");
}
```

3. If it does, add a debug log of your choice, check that your code matches the screenshot below, and hit play:

```
void Start()
{
    FindPartyMember();
}

public void FindPartyMember()
{
    List<string> QuestPartyMembers = new
    List<string>()
    {
        "Grim the Barbarian",
        "Merlin the Wise",
        "Sterling the Knight"
    };

    QuestPartyMembers.Add("Craven the Necromancer");
    QuestPartyMembers.Insert(1, "Tanis the Thief");
    QuestPartyMembers.RemoveAt(0);
    //QuestPartyMembers.Remove("Grim the Barbarian");
```

```
int listLength = QuestPartyMembers.Count;
Debug.LogFormat("Party Members: {0}", listLength);

for(int i = 0; i < listLength; i++)
{
    Debug.LogFormat("Index: {0} - {1}", i,
QuestPartyMembers[i]);

    if(QuestPartyMembers[i] == "Merlin the Wise")
    {
        Debug.Log("Glad you're here Merlin!");
    }
}
```

The console output should look almost the same, except that there is now an extra debug log—one that only printed once when it was Merlin’s turn to go through the loop. More specifically, when *i* was equal to 1 on the second loop, the *if* statement fired and two logs were printed out instead of just one:

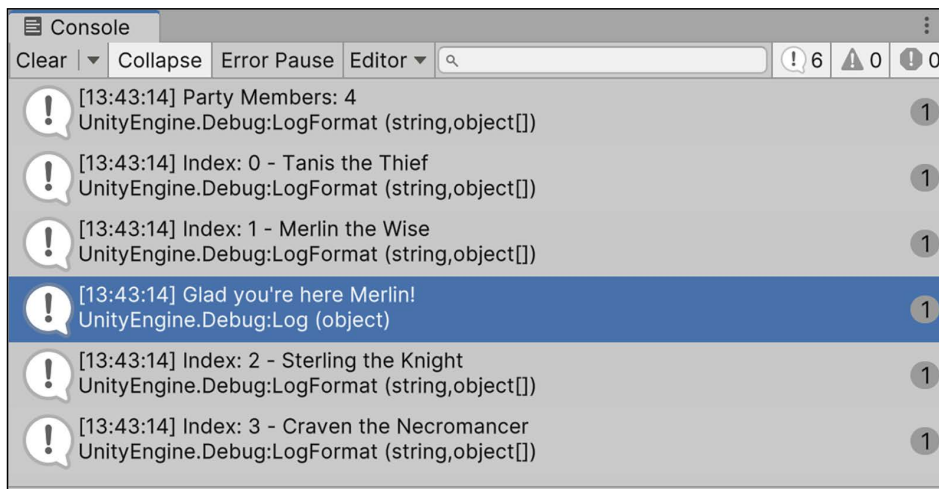


Figure 4.13: Screenshot of the *for* loop printing out list values and matching *if* statements

Using a standard *for* loop can be highly useful in the right situation, but there’s seldom just one way to do things in programming, which is where the *foreach* statement comes into play.

foreach loops

foreach loops take each element in a collection and store each one in a local variable, making it accessible inside the statement. The local variable type must match the collection element type to work properly. foreach loops can be used with arrays and lists, but they are especially useful with dictionaries, since dictionaries are key-value pairs instead of numeric indexes.

In blueprint form, a foreach loop looks like this:

```
foreach(elementType localName in collectionVariable)
{
    code block;
}
```

Let's stick with the QuestPartyMembers list example and do a roll call for each of its elements. Replace the for-loop in FindPartyMembers with the following code:

```
foreach(string partyMember in QuestPartyMembers)
{
    Debug.LogFormat("{0} - Here!", partyMember);
}
```



You can also use the var keyword to automatically determine the type of collection you're looping through, like so:

```
foreach(var partyMember in QuestPartyMembers)
{
    Debug.LogFormat("{0} - Here!", partyMember);
}
```

We can break this down as follows:

- The element type is declared as a string, which matches the values in QuestPartyMembers
- A local variable, called partyMember, is created to hold each element as the loop repeats

- The `in` keyword, followed by the collection we want to loop through, in this case, `QuestPartyMembers`, finishes things off:

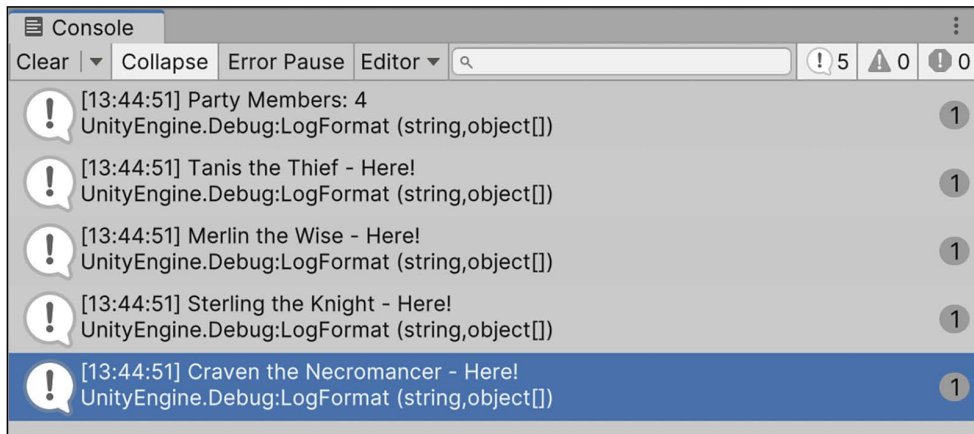


Figure 4.14: Screenshot of a `foreach` loop printing out list values

This is a good deal simpler than the `for` loop. However, when dealing with dictionaries, there are important differences we need to mention—namely how to deal with key-value pairs as local variables.

Looping through key-value pairs

To capture a key-value pair in a local variable, we need to use the aptly named `KeyValuePair` type, assigning both the key and value types to match the dictionary's corresponding types. Since `KeyValuePair` is its type, it acts just like any other element type, as a local variable.

For example, let's loop through the `ItemInventory` dictionary we created earlier in the *Dictionaries* section and debug each key-value like a shop item description:

```
Dictionary<string, int> ItemInventory = new
Dictionary<string, int>()
{
    { "Potion", 5},
    { "Antidote", 7},
    { "Aspirin", 1}
};

foreach(KeyValuePair<string, int> kvp in ItemInventory)
{
    Debug.LogFormat("Item: {0} - {1}g", kvp.Key, kvp.Value);
}
```

We've specified a local variable of `KeyValuePair`, called `kvp`, which is a common naming convention in programming, like calling the `for` loop initializer `i`, and setting the key and value types to `string` and `int` to match `ItemInventory`.

To access the key and value of the local `kvp` variable, we use the `KeyValuePair` properties of `Key` and `Value`, respectively.

In this example, the keys are `strings` and the values are integers, which we can print out as the item name and item price:

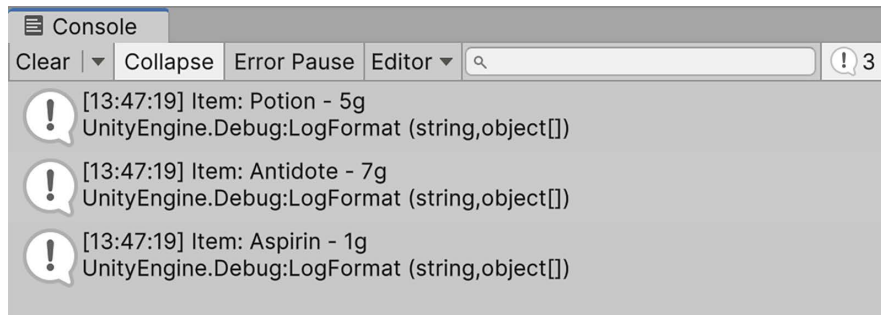


Figure 4.15: Screenshot of a `foreach` loop printing out dictionary key-value pairs

If you're feeling particularly adventurous, try out the following optional challenge to drive home what you've just learned.

Hero's trial—finding affordable items

Using the preceding script, create a variable to store how much gold your fictional character has, and see whether you can add an `if` statement inside the `foreach` loop to check for items that you can afford.



Hint: use `kvp.Value` to compare prices with what's in your wallet.

while loops

`while` loops are similar to `if` statements in that they run as long as a single expression or condition is true.

Value comparisons and Boolean variables can be used as `while` conditions, and they can be modified with the `NOT` operator.

The while loop syntax says, *While my condition is true, keep running my code block indefinitely:*

```
Initializer
while (condition)
{
    code block;
    iterator;
}
```

With while loops, it's common to declare an initializer variable, as in a for loop, and manually increment or decrement it at the end of the loop's code block. We do this to avoid an infinite loop, which we will discuss at the end of the chapter. Depending on your situation, the initializer is usually part of the loop's condition.

while loops are very useful when coding in C#, but they are not considered good practice in Unity because they can negatively impact performance and routinely need to be manually managed.

Let's take a common use case where we need to execute code while the player is alive, and then debug when that's no longer the case:

1. Create a public variable called `PlayerLives` of the `int` type and set it to 3:

```
public int PlayerLives = 3;
```

2. Create a new public function called `HealthStatus`:

```
public void HealthStatus()
{
}
```

3. Declare a while loop with the condition checking whether `PlayerLives` is greater than 0 (that is, the player is still alive):

```
while(PlayerLives > 0)
{
}
```

4. Inside the while loop, debug something to let us know the character is still kicking, then decrement `PlayerLives` by 1 using the `--` operator:

```
Debug.Log("Still alive!");
PlayerLives--;
```

5. Add a debug log after the while loop curly brackets to print something when our lives run out:

```
Debug.Log("Player KO'd...");
```

6. Call the `HealthStatus` method inside `Start`:

```
void Start()
{
    HealthStatus();
}
```

Your code should look like the following:

```
public int PlayerLives = 3;

void Start()
{
    HealthStatus();
}

public void HealthStatus()
{
    while(PlayerLives > 0)
    {
        Debug.Log("Still alive!");
        PlayerLives--;
    }

    Debug.Log("Player KO'd...");
}
```

With `PlayerLives` starting out at 3, the while loop will execute three times. During each loop, the debug log, "Still alive!", fires, and a life is subtracted from `PlayerLives`.

When the while loop goes to run a fourth time, our condition fails because `PlayerLives` is 0, so the code block is skipped and the final debug log prints out:

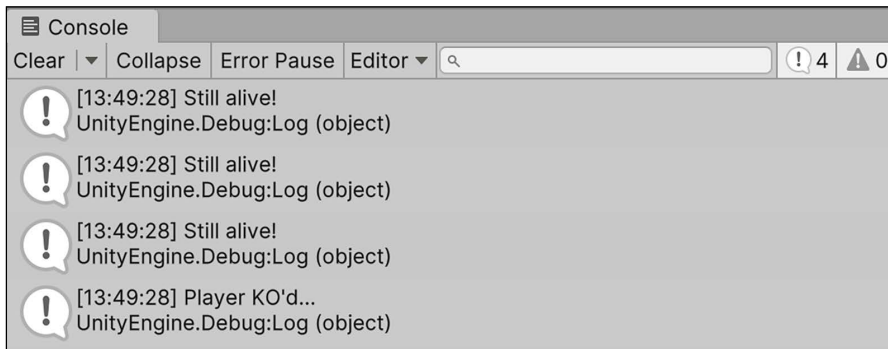


Figure 4.16: Screenshot of while loop output in the console

If you're not seeing multiple "Still alive!" debug logs, make sure the **Collapse** button in the **Console** toolbar isn't selected:

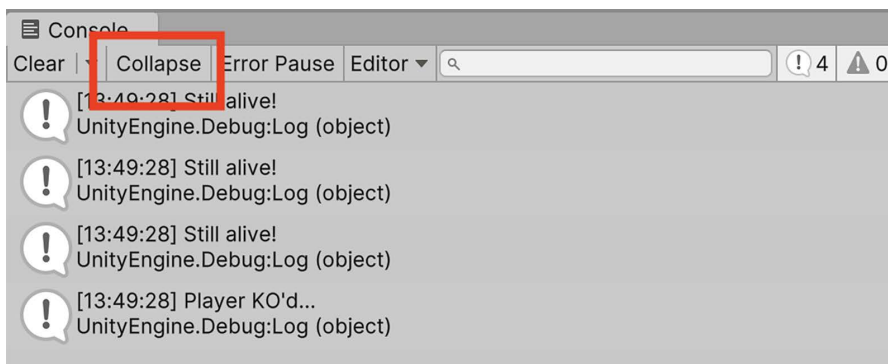


Figure 4.17: Screenshot of console messages with Collapse option unchecked

The question now is what happens if a loop never stops executing? We'll discuss this issue in the following section.

To infinity and beyond

Before finishing this chapter, we need to understand one extremely vital concept when it comes to iteration statements: *infinite loops*. These are exactly what they sound like: when a loop's conditions make it impossible for it to stop running and move on in the program. Infinite loops usually happen in for and while loops when the iterator is not increased or decreased; if the `PlayerLives` line of code was left out of the while loop example, Unity would freeze and/or crash, recognizing that `PlayerLives` would always be 3 and executing the loop forever.

Iterators are not the only culprits to be aware of; setting conditions in a `for` loop that will never fail, or evaluate to `false`, can also cause infinite loops. In the party members example, from the *Looping through key-value pairs* section, if we had set the `for` loop condition to `i < 0` instead of `i < QuestPartyMembers.Count`, `i` would always be less than 0, looping until Unity crashed.

Summary

As we bring the chapter to a close, we should reflect on how much we've accomplished and what we can build with that new knowledge. We know how to use simple `if-else` checks and more complex `switch` statements, allowing decision making in code. We can create variables that hold collections of values with arrays and lists or key-value pairs with dictionaries.

This allows complex and grouped data to be stored efficiently. We can even choose the right looping statement for each collection type, while carefully avoiding infinite-loop crashes.

If you're feeling overloaded, that's perfectly OK—logical, sequential thinking is all part of exercising your programming brain.

The next chapter will complete the basics of C# programming with a look at classes, structs, and **object-oriented programming (OOP)**. We'll be putting everything we've learned so far into these topics, preparing for our first real dive into understanding and controlling objects in the Unity engine.

Unlock this book's exclusive benefits now

This book comes with additional benefits designed to elevate your learning experience.



Note: Have your purchase invoice ready before you begin. <https://www.packtpub.com/unlock/9781837636877>

